GOVERNMENT OF MADHYA PRADESH • PARAKH INSPECTION SYSTEM

Aadhaar-Based eSign Technical Specification & Extension Manual

Comprehensive architectural guide, cryptographic specifications, live gateway screenshots, and step-by-step developer implementation patterns for extending Aadhaar eSign (MPSeDC / C-DAC Hastakshar) across all modules in Parakh.

Version: 1.0 (Production) | **Gateway:** MPSeDC / C-DAC Hastakshar 2.0 | **Crypto Standard:** PKCS#7 Detached • XMLDSIG | **Backend:** .NET 6.0 Web API | **Frontend:** Angular 14+

1. System Overview & Objective

The **Parakh Inspection Portal** integrates with the official **MPSeDC / C-DAC Hastakshar Aadhaar eSign Gateway** to facilitate legally valid, tamper-proof electronic signatures under the *Information Technology Act, 2000 (Section 3A)*.

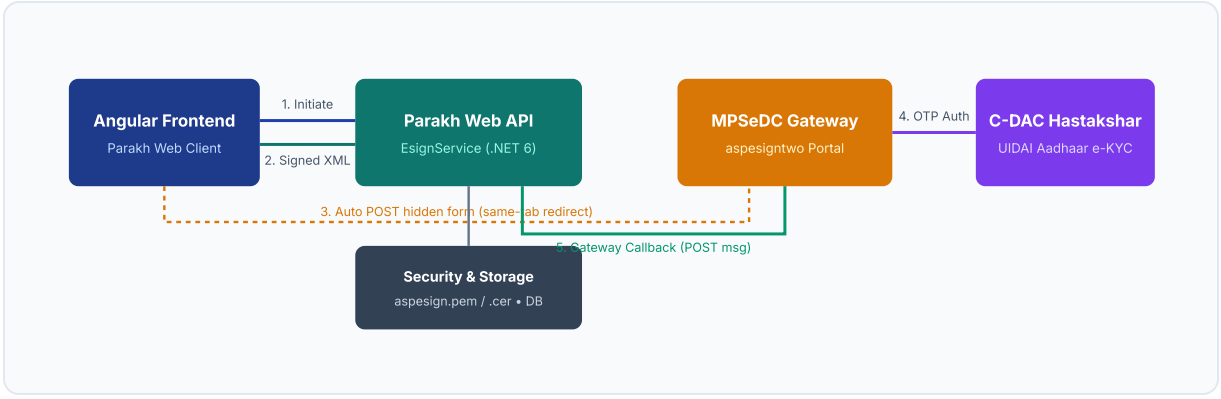
This integration eliminates the physical requirement of signing inspection reports, allocation orders, and compliance certificates manually with pen-and-paper. Government authorities authenticate themselves directly using Aadhaar One-Time Password (OTP) or Biometric credentials. Upon successful identity verification, a certified digital certificate is dynamically generated by C-DAC's Certifying Authority (e-Mudhra / C-DAC ESP) and cryptographically embedded into the PDF document.

Document Objective:
This technical document provides the complete structural blueprint of the eSign service currently operating in Parakh, explains how the cryptographic handshake functions, presents the actual gateway flow screens, and delivers a reusable copy-paste recipe for any developer to extend eSign to new modules (e.g. Labour Inspections, Mandir Samagra, Shop Act Orders, etc.).

2. Architecture & Security Topology

The eSign subsystem is designed with strict data privacy principles: **the actual contents of the PDF document NEVER leave the Parakh server**. Only an asymmetric cryptographic hash (SHA-256) of the document byte-range is transmitted to the eSign gateway inside a cryptographically signed XML document.

High-Level System Architecture



Core Cryptographic Components

Component	Standard / Format	Technical Responsibility in Parakh
XMLDSIG	W3C XML Digital Signature (RSA-SHA256)	Signs the outgoing `AspEsignReq` XML using MPSeDC department private key (`aspesign.pem`). Guarantees request authenticity to the gateway.
Gateway Verification	X.509 Public Certificate (`aspesign.cer`)	Validates the signature on the incoming `AspEsignResp` XML during the gateway callback to prevent spoofing or man-in-the-middle attacks.
Range Stream Hashing	SHA-256 (256-bit Digest)	Calculated directly over the unclosed PDF byte stream. Guarantees that even 1 byte altered after signing invalidates the signature.
PKCS#7 Detached Seal	Adobe PPKlite (`ADBE_PKCS7_DETACHED`)	The actual digital certificate returned by C-DAC in the `DocSignature` XML element, injected into the reserved 1MB space inside the PDF structure.
Signatory Identity	X.509 Subject DN • OID 2.5.4.45 / 2.5.4.3	Extracts the signer's legally verified Common Name (CN) and Aadhaar UID Token from the certificate chain using BouncyCastle.

3. The 6-Stage End-to-End Signing Lifecycle

The complete end-to-end execution path consists of 6 chronological phases from the time the user clicks *"Schedule & eSign"* in the UI to the final archival of the digitally signed PDF.

1

Draft PDF & Stamper

PDF loaded into `PdfStamper`. Predefined corners or keyword search executes to calculate signature rectangle coordinates. Reserved 1MB placeholder allocated.

2

SHA-256 & XMLDSIG

SHA-256 hash computed over PDF range stream. `AspEsignReq` XML generated and signed with `aspesign.pem`. Active session cached in memory.

3

Gateway Form POST

Angular client receives Gateway URL & Base64 XML. Automatically renders and submits hidden HTML form. User redirects seamlessly to MPSeDC.

4

Aadhaar Consent & OTP

Signer grants consent on MPSeDC, enters Aadhaar number, requests OTP, and submits OTP on C-DAC Hastakshar portal for instant e-KYC.

5

Callback & PKCS#7 Injection

Gateway invokes Parakh backend callback. Server verifies XMLDSIG with `aspesign.cer`,

6

Database Update & Redirect

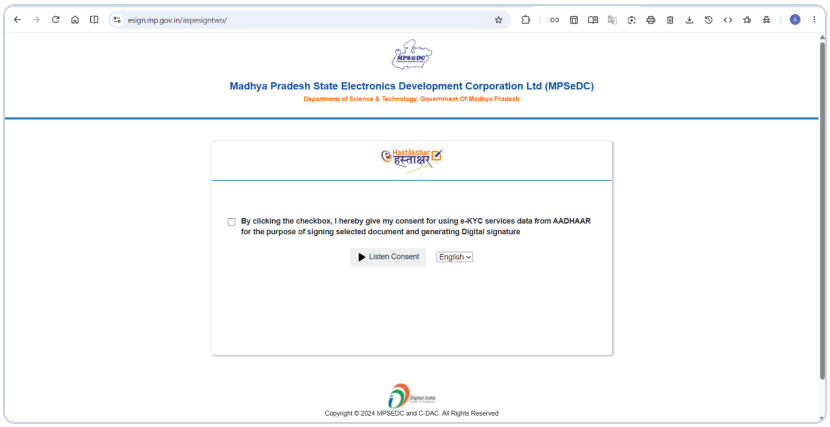
Backend updates database table (order number, signed file path). Returns HTML redirect back to Angular portal with success parameters.

extracts `DocSignature`, and injects PKCS#7 bytes into PDF.

4. Live Gateway Workflow & Screens

The following screenshots document the actual user-facing portal flow provided by the MP Government eSign integration:

Step 1: MPSeDC Hastakshar Consent Portal (esign.mp.gov.in/aspesigntwo) — Redirect Step



User Action: Once redirected from Parakh, the user reviews the Aadhaar e-KYC consent statement. Clicking the checkbox and proceeding redirects the browser to the C-DAC Hastakshar Aadhaar authentication engine.

Step 2: C-DAC Hastakshar Aadhaar OTP Verification Screen — Authentication Step



User Action: The user enters their 12-digit Aadhaar Number (or VID / UID Token), clicks *"Get OTP"*, enters the 6-digit OTP received on their Aadhaar-linked mobile phone, checks the authentication consent box, and clicks *"Submit"*.

5. Existing Codebase Implementation

The eSign subsystem is architected in an extensible, decoupled manner inside the Parakh repository:

Layer / File Path	Primary Class / Component	Description
`ParakhAPI/Utility/EsignService.cs`	`IEsignService`, `EsignService`	Core cryptographic engine. Handles PDF manipulation, keyword coordinate search, SHA-256 calculation, XMLDSIG generation, gateway response parsing, and PKCS#7 detached signature injection.
`ParakhAPI/LabourInspectionAPI/Controllers/CoopprativeController.cs`	`CoopprativeController`	Exposes endpoints `InitiateRoosterEsign`, `EsignCallback`, and `GetEsignStatus`. Orchestrates DB updates and handles gateway redirect.
`ParakhAPI/LabourInspectionAPI/appsettings.json`	`"ESign": Ellipsis`	Centralized configuration block containing Gateway URL, Department ID, Secret Key, Application ID, Certificate Paths, and Callback URL.
`Parakh/src/app/super-admin/mp-cooprative/add-rooster/add-rooster.component.ts`	`AddRoosterComponent`	Angular component that calls `masterService.InitiateRoosterEsign`, executes `submitToEsignGateway()`, and listens for query parameters on return.

Configuration Settings (appsettings.json)

appsettings.json — ESign Section

```
"ESign": {
  "GatewayUrl": "https://esign.mp.gov.in/aspesigntwo/",
  "DepartmentId": "_Department_ID",
  "SecretKey": "SHA_Key",
  "ApplicationId": "_Application_ID",
  "PemPath": "Certificates/aspesign.pem",
  "CerPath": "Certificates/aspesign.cer",
  "ResponseUrl": "https://<YOUR_API_DOMAIN>/api/Cooprative/EsignCallback"
}
```

6. Flexible Signature Placement Engine

A major feature implemented in `EsignService.cs` is the **multi-mode visual signature placement engine**. Developers can position the signature stamp anywhere on the document without hardcoding pixel values:

1. Keyword-Based Placement

Scans every page for a target keyword (supports Hindi Unicode such as "सहकारी संस्थाएँ मध्यप्रदेश"). Automatically calculates the bounding box and stamps the seal above, below, or on top of the text.

```
Position = EsignPositionType.KeywordBased
```

2. Predefined Corner Anchors

Positions the signature in standard document quadrants: `BottomRight`, `BottomLeft`, `TopRight`, or `TopLeft` with customizable margins (e.g. 25px horizontal, 20px vertical).

```
Position = EsignPositionType.BottomRight
```

3. Custom (X, Y) Coordinates

For documents with fixed-coordinate form templates. Allows exact specification of `CustomX`, `CustomY`, `BoxWidth`, and `BoxHeight` in PDF points.

```
Position = EsignPositionType.CustomXY
```

Page Selection Options

- `EsignPageSelection.AllPages` — Signs every single page of the document (standard for regulatory inspection orders).
- `EsignPageSelection.LastPage` — Signs only the closing signature block on the final page.
- `EsignPageSelection.FirstPage` — Signs cover letters or initial approval notices.
- `EsignPageSelection.SpecificPage` — Target a specific page index via `TargetPageNumber`.

7. Developer Extension Guide (Step-by-Step Cookbook)

To enable eSign on any new feature, document, or controller in Parakh, follow this simple 3-step integration recipe:

Step 1: Inject IEsignService into your Controller

NewController.cs — Constructor Injection

```
public class LabourInspectionController : ControllerBase
{
    private readonly IEsignService _esignService;
    private readonly IConfig _config;

    public LabourInspectionController(IEsignService esignService, IConfig config)
    {
        _esignService = esignService;
        _config = config;
    }
}
```

Step 2: Create the Initiation Endpoint

Build the `EsignRequest` object with your module's specific PDF path, entity reference ID, placement coordinates, and visual typography:

LabourInspectionController.cs — InitiateEsign Action

```
[HttpPost("InitiateInspectionEsign")]
public async Task<IActionResult> InitiateInspectionEsign([FromBody] InitiateInspectionEsignDTO input)
{
    // 1. Generate or fetch draft inspection report PDF path
    string relativePdfPath = $"InspectionReports/Report_{input.InspectionId}.pdf";
    string fullInputPdfPath = Path.Combine(_config.FilePath, relativePdfPath);

    // 2. Configure EsignRequest
    var esignRequest = new EsignRequest
    {
        InputPdfPath = fullInputPdfPath,
        RelativeDocPath = relativePdfPath,
        ReferenceId = input.InspectionId.ToString(),
        UserId = input.UserId,
        SignerName = input.OfficerName,
        Designation = input.OfficerDesignation,
        OrderNumber = input.InspectionNumber,
        Remark = "Inspected and digitally signed under Labour Laws",
```

```
        // Routing endpoints
        ResponseUrl = $"{_config.Parakh_url}/api/LabourInspection/InspectionEsignCallback",
        ClientReturnUrl = input.ReturnUrl,

        // Placement configuration
        Placement = new EsignPlacementOptions
        {
            Position = EsignPositionType.BottomRight,
            PageSelection = EsignPageSelection.AllPages,
            MarginX = 25f,
            MarginY = 20f,
            BoxWidth = 190f,
            BoxHeight = 65f
        },

        // Visual stamp typography
        Design = new EsignDesignOptions
        {
            Fields = new Dictionary<string, string>
            {
                ["Name"] = input.OfficerName,
                ["Designation"] = input.OfficerDesignation,
                ["Inspection No"] = input.InspectionNumber
            },
            IncludeDate = true,
            DateFormat = "dd/MM/yyyy HH:mm",
            FontSize = 8f
        }
    };

    // 3. Execute eSign Initiation
    var result = _esignService.InitiateEsign(esignRequest);

    if (result.IsSuccess)
    {
        return Ok(new { isSuccess = true, data = result });
    }
    return BadRequest(new { isSuccess = false, message = result.Message });
}
```

Step 3: Create the Gateway Callback Endpoint

Receives the asynchronous signed token from the gateway, triggers PKCS#7 PDF injection via `_esignService.ProcessGatewayCallback`, updates your database, and redirects the user back to the Angular portal:

LabourInspectionController.cs — Callback Action

```
[HttpPost("InspectionEsignCallback")]
[HttpGet("InspectionEsignCallback")]
public async Task<IActionResult> InspectionEsignCallback()
{
    // 1. Read 'msg' parameter from Form POST or Query string
    string msg = Request.HasFormContentType && Request.Form.ContainsKey("msg")
        ? Request.Form["msg"].ToString()
        : Request.Query["msg"].ToString();

    // 2. Process gateway XML verification & PDF signature stamping
    var callbackResult = _esignService.ProcessGatewayCallback(msg);
    var context = _esignService.GetStatus(callbackResult.TransId);

    string returnUrl = context?.ReturnUrl ?? $"{_config.Parakh_url}/InspectionDashboard";
    char sep = returnUrl.Contains('?') ? '&' : '?';

    if (callbackResult.IsSuccess)
    {
        // 3. Update module-specific database records
```

```
        await _inspectionRepo.UpdateSignedStatusAsync(
            inspectionId: Convert.ToInt64(callbackResult.ReferenceId),
            signedDocPath: callbackResult.SignedDocPath,
            signatoryName: callbackResult.SignatoryName,
            uidToken: callbackResult.SignatoryUidToken
        );

        // 4. Auto-redirect browser back to Angular UI
        string redirectSuccessUrl = $"{returnUrl}{sep}esign=success&refId={callbackResult.ReferenceId}";
        return Content($@"<!DOCTYPE html><html><head>
            <meta http-equiv='refresh' content='1;url={redirectSuccessUrl}'></head>
            <body>Signing successful! Redirecting to Parakh...</body></html>", "text/html");
    }
    else
    {
        string redirectFailUrl = $"{returnUrl}{sep}esign=failed&error={Uri.EscapeDataString(callbackResult.Message)}";
        return Content($@"<!DOCTYPE html><html><head>
            <meta http-equiv='refresh' content='3;url={redirectFailUrl}'></head>
            <body>Signing failed: {callbackResult.Message}. Redirecting...</body></html>", "text/html");
    }
}
```

Step 4: Frontend Angular Submission Helper

In your Angular component, trigger the gateway redirect using the standard form POST helper:

your-feature.component.ts — submitToEsignGateway

```
submitToEsignGateway(gatewayUrl: string, signedXmlMsg: string) {
  const form = document.createElement('form');
  form.method = 'POST';
  form.action = gatewayUrl;
  form.target = '_self'; // Full-page same tab redirect (eliminates popup blocker issues)

  const inputMsg = document.createElement('input');
  inputMsg.type = 'hidden';
  inputMsg.name = 'msg';
  inputMsg.value = signedXmlMsg;
  form.appendChild(inputMsg);

  document.body.appendChild(form);
  form.submit();
}
```

8. API & Payload Reference

AspEsignReq (Outgoing XML to Gateway)

Sample AspEsignReq XML

```
<?xml version="1.0" encoding="utf-8"?>
<AspEsignReq transId="a3f9104b901a4e5bb3889100fa123456"
             authMode="1"
             responseUrl="https://<YOUR_API_DOMAIN>/api/Cooprative/EsignCallback">
  <ESignInput>
    <applicationID>_Application_ID</applicationID>
    <departmentID>_Department_ID####SHA_Key</departmentID>
    <docHash>
      8f52b61ef0a29583b63ec85a213941a382c40398246991307ec32f153096057a
    </docHash>
    <docName>Cooperative_Order_1042.pdf</docName>
    <docInfo></docInfo>
    <remark>Signed by Authority</remark>
    <aadhaarLast4Digits>1234</aadhaarLast4Digits>
```

```
    <uidToken></uidToken>
  </ESignInput>
  <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
    <SignedInfo>...</SignedInfo>
    <SignatureValue>[RSA-SHA256 Base64]</SignatureValue>
  </Signature>
</AspEsignReq>
```

AspEsignResp (Incoming XML from Gateway Callback)

Sample AspEsignResp XML

```
<?xml version="1.0" encoding="utf-8"?>
<AspEsignResp status="1" errCode="NA" errMsg="NA" resCode="1"
              transId="a3f9104b901a4e5bb3889100fa123456">
  <UserSignatures>
    <DocSignature id="1">
      MIAGCSqGSIb3DQEHAqCAMIACAQExDzANBglghkgBZQMEAgEFADCABgkqhkiG9w0BBwEAAKCC...
      [PKCS#7 Detached Signature Value]
    </DocSignature>
  </UserSignatures>
  <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">...</Signature>
</AspEsignResp>
```

9. Troubleshooting & Error Matrix

Error / Scenario	Root Cause	Resolution / Corrective Action
`Signature keyword 'XYZ' was not found`	`KeywordBased` placement mode was chosen but the exact string does not appear in the generated draft PDF.	Verify spelling and encoding. If using Hindi fonts, ensure the PDF generation engine outputs selectable Unicode text (not outlines/bitmaps). Or set fallback to `BottomRight`.
`No active signing session found for transaction [ID]`	Active signing sessions expire after 30 minutes, or the backend application pool recycled while the user was on the gateway.	Ask the user to restart the eSign flow. In multi-instance load-balanced web farms, ensure sticky sessions are enabled or persist context in a shared Redis cache.
`Response XML signature verification failed`	Gateway signature does not match public certificate at `ESign:CerPath`.	Ensure the latest `aspesign.cer` from MPSeDC is placed in the `Certificates/` directory and not expired.
`Invalid Rooster / Rooster Already Scheduled`	Duplicate submission attempt after a roster was already signed.	Controller blocks duplicate calls. Refresh roster list to verify current status.
`Verification certificate not found at Certificates/aspesign.cer`	Relative path resolution issue during deployment.	`EsignService.ResolveCertificatePath()` automatically probes CWD, BaseDirectory, and `Certificates/` folder. Ensure certificates are marked *"Copy to Output Directory = Always"* in the `.csproj` file.

10. Security & Compliance Standards

- **Zero Aadhaar Storage:** Neither Aadhaar number nor biometric samples ever touch the Parakh application or database. Only the anonymized `SignatoryUidToken` is retained for audit logging.
- **Cryptographic Isolation:** The PDF body is never transmitted across the network to MPSeDC or C-DAC; only the SHA-256 digest is communicated.
- **Tamper-Evident Verification:** Any post-signing alteration to the PDF invalidates the cryptographic hash and causes Adobe Acrobat Reader to display a red signature failure banner.
- **Legal Recognition:** Complies with Rule 3 of the Information Technology (Electronic Service Delivery) Rules and Second Schedule of the Information Technology Act, 2000.